

四、模型量化

参考：

<https://www.53ai.com/news/LargeLanguageModel/2024060272389.html>

LLM 模型量化(Quantization)是一种通过降低模型权重和激活值/模型运算的输出值的数值精度，来减少模型存储体积、降低计算资源并提升推理速度的技术，同时要求尽可能保留模型的性能。是一种解决 LLM 部署成本高、硬件门槛高的核心手段。

一、核心原理

LLM 训练的参数格式一般是 32 位浮点数(FP32)。量化本质就是**将高精度数值(FP32)映射到低精度格式(eg:INT8)**，通过牺牲部分数值精度，来换取存储和计算效率的提升。

比如：

- FP32 → INT8：参数体积减少 75% 左右，计算速度提升 3-4 倍左右；

二、常见量化方式

1. 线性量化

- 直接通过线性映射将高精度的数据压缩到低精度范围；
- 公式：量化值=(原始值 - 零点) / 缩放因子；零点和缩放因子一般是从参数中统计出来的，该方式类似归一化处理；
- 优点：实现简单，计算高效；
- 缺点：对异常值敏感，可能会导致比较大的精度损失；

2. 非线性量化

- 类似线性量化做映射操作，但是映射公式需要特定设计，比如采用对数映射函数；
- 优点：相比于线性量化能够保留更多的精度信息；
- 缺点：实现复杂，开销比较大；

3. 动态量化

- 推理时对激活输出值实时量化(模型参数权重提前量化)，根据输入数据动态调整量化相关参数；
- 优点：适应不同的输入分布，损失较小；
- 缺点：实时计算增加部分开销；

4. 静态量化

- 提前用校准数据 (Calibration Data) 计算量化参数 (缩放因子、零点), 推理时直接使用固定参数量化。
- 优点: 推理速度快, 无实时计算开销;
- 缺点: 依赖校准数据的代表性, 分布偏移时精度下降。

三、常见数值类型

所有浮点数(二进制)均遵循 IEEE754 标准, 主要由三部分组成:

- 符号位(S): 1 位, 表示正负(0 表示正, 1 表示负)
- 指数位(E): 决定数值的“动态范围”(可表示的最大值和最小值), 位数越多, 动态范围越大。
- 尾数位(M): 决定数值的“精度”(小数部分的精确程度), 位数越多, 精度越高。

常见类型:

FP32: 32 位(1+8+23)组成, 精度 7~8 位, 标准单精度浮点数, 精度高, 但是存储成本也高;

FP16: 16 位(1+5+10)组成, 精度 3~4 位, 半精度浮点数, 存在小值无法表达的风险;

BFP16: 16 位(1+8+7)组成, 精度 2~3 位, 脑浮点数, 范围基本和 FP32 一致, 精度低于 FP16;

FP8: 8 位(1+4+3 或者 1+5+2)组成, 精度 1~2 位, 精度及范围进一步压缩;

INT8: 8 位(1+7)组成, 取值范围 $[-128, 127]$, 整数无小数精度, 动态范围及精度更低, 单计算速度快、支持的硬件平台广泛;

四、Transformer 的模型量化方式

可直接采用 transformers 库进行量化处理

Python

```
## 仅供参考

from transformers import BitsAndBytesConfig
bnb_config = BitsAndBytesConfig(
    load_in_8bit=True, # 权重INT8量化
    llm_int8_threshold=6.0 # 激活值动态量化的阈值 (超过此值用FP16计算)
)
# 模型恢复&量化处理
model = AutoModelForCausalLM.from_pretrained(
    model_id, # 模型id字符串
    quantization_config=bnb_config,
    device_map="auto"
)

##### 此时的model就是量化后的模型, 可以进行持久化保存以及推理应用等操作
```